

ISB206 Veri Madenciliği

7. Ders: Naive Bayes Uygulama

01.04.2026

1. Uygulama: Ürün verisi

```
> rm(list=ls())  
> library(naivebayes) # Naive bayes çalıştırmak için gerekli  
> library(caret) # sınıflama performansı ölçebilmek için gerekli  
# eğer kütüphaneler bilgisayarda yüklü değilse  
  
# install.packages("kütüphaneadı") komutunu çalıştır.  
> data<-read.csv("urun.csv") # Hata veriyorsa veri dosyasını  
# doğru klasöre taşı
```

1. Uygulama: Ürün verisi

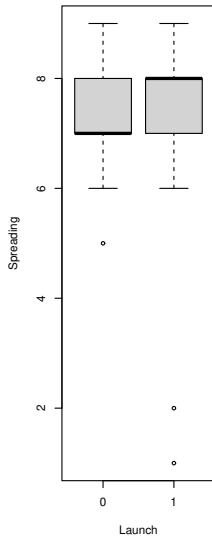
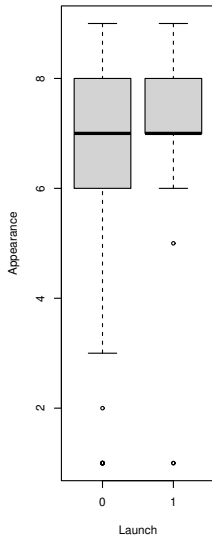
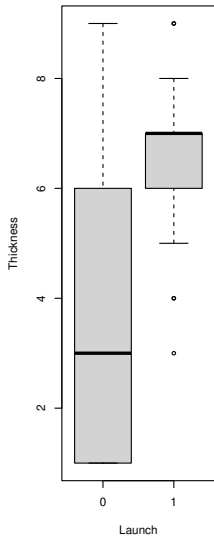
```
> rm(list=ls())
> library(naivebayes) # Naive bayes çalıştırmak için gerekli
> library(caret) # sınıflama performansı ölçebilmek için gerekli
# eğer kütüphaneler bilgisayarda yüklü değilse

# install.packages("kütüphaneadı") komutunu çalıştır.
> data<-read.csv("urun.csv") # Hata veriyorsa veri dosyasını
                             # doğru klasöre taşı

> str(data)
data.frame': 95 obs. of 5 variables:
 $ Launch: int 0 0 0 0 0 0 0 0 0 0 ...
 $ Thickness: int 1 1 1 1 1 1 1 1 1 1 ...
 $ Appearance: int 9 8 7 8 8 7 9 7 9 9 ...
 $ Spreading: int 8 7 7 9 7 7 8 7 9 8 ...
 $ Rank: int 2 1 1 1 1 1 2 2 1 2 ...
```

Veri Ön İnceleme

```
> par(mfrow=c(1,3))  
> boxplot(Thickness ~ Launch,data=data)  
> boxplot(Appearance ~ Launch,data=data)  
> boxplot(Spreading ~ Launch,data=data)  
> par(mfrow=c(1,1))
```



Eğitim ve Test Verisi Ayrıştırma

```
> nrow(data)*0.80 #Eğitim verisinde kaç gözlem olmalı  
[1] 76  
orn<-sample(1:95,76,replace=F) # 1:95 vektöründen 76 tane  
                                # rasgele sayı seçer
```

Eğitim ve Test Verisi Ayırıştırma

```
> nrow(data)*0.80 #Eğitim verisinde kaç gözlem olmalı
[1] 76
orn<-sample(1:95,76,replace=F) # 1:95 vektöründen 76 tane
                                # rasgele sayı seçer

# Sınıf değişkenini eğitim ve test verisi olarak böler
> data_train_class<-as.factor(data[orn,1])
> data_test_class<-as.factor(data[-orn,1])

# Kalan veriyi eğitim ve test verisi olarak böler
> data_train<-data[orn,-1]
> data_test<-data[-orn,-1]
```

Model uydurma ve tahmin

```
> model<-naive_bayes(x=data_train,y=data_train_class,laplace = T)

# Model eğitim verisinde iyi performans gösterdi mi?
> tahmin_train<-predict(model, data_train)
> confusionMatrix(tahmin_train, data_train_class)

# Model test verisinde iyi performans gösterdi mi?
> tahmin_test<-predict(model, data_test)
> confusionMatrix(tahmin_test, data_test_class)
```


2. Örnek Spam SMS

```
>rm(list=ls())  
# Gerekli kütüphaneler  
> library(naivebayes)  
> library(tm) # text mining için gerekli  
> library(SnowballC) # kelimeleri kısaltmak için gerekli  
> library(wordcloud) # kelime bulutu oluşturmak için gerekli
```

2. Örnek Spam SMS

```
>rm(list=ls())
# Gerekli kütüphaneler
> library(naivebayes)
> library(tm) # text mining için gerekli
> library(SnowballC) # kelimeleri kısaltmak için gerekli
> library(wordcloud) # kelime bulutu oluşturmak için gerekli

> sms<-read.csv("sms_spam.csv")
> str(sms)
'data.frame': 5559 obs. of 2 variables:
 $ type: chr "ham" "ham" "ham" "spam" ...
 $ text: chr "Hope you are having a good week. Just checking
in" "K..give back my thanks." "Am also doing in cbe only.
But have to pay." "complimentary 4 STAR Ibiza."
# Encoding(sms$text) <- "UTF-8" İlerde UTF-8 Hatası verirse
aktive et
```

Veri Ön İnceleme

```
>table(sms$type)  
ham spam  
4812 747
```

Veri Ön İnceleme

```
# Mesajlar corpus tipine dönüştürülür
> sms_corpus <- VCorpus(VectorSource(sms$text))
# Corpus tipinde metin verileri kolaylıkla görüntülenebilir.
> as.character(sms_corpus[[1]])
```

Veri Ön İnceleme

```
# Mesajlar corpus tipine dönüştürülür
> sms_corpus <- VCorpus(VectorSource(sms$text))
# Corpus tipinde metin verileri kolaylıkla görüntülenebilir.
> as.character(sms_corpus[[1]])
# Tüm harfleri küçültülür
> sms_corpus_clean <- tm_map(sms_corpus,
                             content_transformer(tolower))

# Rakamları siler
> sms_corpus_clean <- tm_map(sms_corpus_clean, removeNumbers)
# Gereksiz kelimeleri siler
> sms_corpus_clean <- tm_map(sms_corpus_clean, removeWords,
                             stopwords())

# Noktalamaları siler
> sms_corpus_clean <- tm_map(sms_corpus_clean, removePunctuation)
# Kelimleri eksiz hale dönüştürür
> sms_corpus_clean <- tm_map(sms_corpus_clean, stemDocument)
```

Öncesi ve Sonrası

```
> as.character(sms_corpus[[15]])  
[1] "K I'll be sure to get up before noon and see what's  
what"  
  
> as.character(sms_corpus_clean[[15]])  
[1] "k sure get noon see"
```

Kelime bulutu

```
> wordcloud(sms_corpus_clean, min.freq = 50, random.order =  
FALSE)
```


Doküman matrisi oluşturma ve eğitim-test verisi ayırma

```
#Döküman matrisi oluşturma
> sms_dtm<-DocumentTermMatrix(sms_corpus_clean)

> sms_dtm_train <- sms_dtm[1:4169, ]
> sms_dtm_test <- sms_dtm[4170:5559, ]

# Sınıf değişkenleri test-eğitim ayırma
> sms_train_labels <- sms[1:4169, ]$type
> sms_test_labels <- sms[4170:5559, ]$type
> prop.table(table(sms_train_labels))
> prop.table(table(sms_test_labels))
```

Verinin son düzenlenmesi

```
# Beşten fazla geçen kelimelerin bulunması
> sms_freq_words <- findFreqTerms(sms_dtm_train, 5)
> sms_dtm_freq_train <- sms_dtm_train[ , sms_freq_words]
> sms_dtm_freq_test <- sms_dtm_test[ , sms_freq_words]

# Kelime sayılarının varlık yokluğa dönüştürülmesi
> convert_counts <- function(x) {
x <- ifelse(x > 0, "Yes", "No")
}

> sms_train <- apply(sms_dtm_freq_train, 2, convert_counts)
> sms_test <- apply(sms_dtm_freq_test, 2, convert_counts)
```

Model uydurma ve performns ölçüleri

```
# Model
> sms_classifier <- naive_bayes(sms_train,
sms_train_labels,laplace=1)
# tahmin verisi analiz sonucu
> tahmin_train<-predict(sms_classifier,sms_train)
> confusionMatrix(tahmin_train, as.factor(sms_train_labels))

# test verisi analiz sonucu
> tahmin_test<-predict(sms_classifier,sms_test)
> confusionMatrix(tahmin_test, as.factor(sms_test_labels))
```